arXiv:2611.14387v1 [cs.CL] 10 Nov 2026

Blurring the Block Boundaries: Overlapping Block Denoising for Efficient Diffusion Language Models

Amir Mann[1], Jonathan Fhima[2], Shaked Brody[2], Or Avitan[2], Asif Zamir[2], Shahar Tsiper[2], Niv Nayman[2]

Abstract

Diffusion Language Models (DLMs) have emerged as effective alternatives to autoregressive models for latency-critical tasks, enabling faster token generation. Recent advances employ block diffusion with self-editing capabilities, where each block refines tokens iteratively before advancing to the next one. Yet each block is only as fast as its slowest token, and the next block cannot start before it. We suggest Blurring the Block Boundaries (*B3*), a novel method with which two blocks are refined in parallel within a shared forward pass. *B3* calibrates itself on the fly, opening the next block only once the model is ready to start generating it. It is a training-free and hyperparameter-free acceleration method which seamlessly integrates within any self-editing block diffusion model. Evaluated on state-of-the-art DLMs across 10 diverse datasets spanning varied tasks and difficulty levels, *B3* results in on par generation quality and 11.5–14.5% speedup compared to the base model, pushing the Pareto frontier for latency–quality trade-offs in DLM inference.

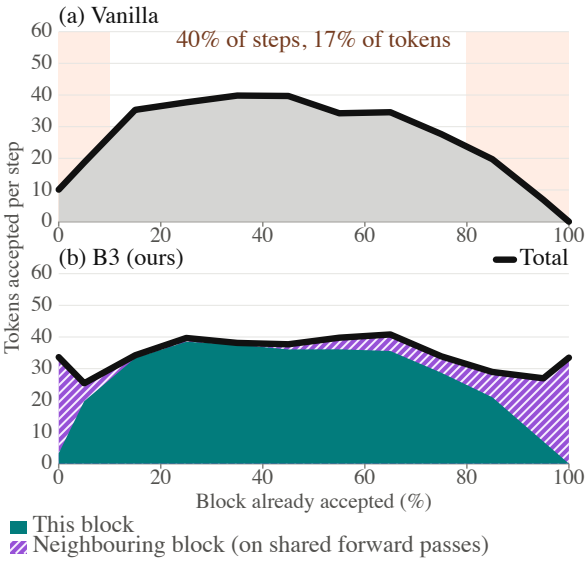

Figure 1: Tokens accepted per denoising step against the share of the block already accepted, for DiffusionGemma 26B on HumanEval+. (a) Vanilla decoding: the orange bands mark low-acceptance regions at the block's boundaries, which take 40% of the steps but yield only 17% of the tokens. (b) *B3*: forward passes shared with the neighbouring block (purple) keep the tokens accepted per step consistent across the block.

1 Introduction

Discrete Diffusion Language Models (DLMs) (Campbell et al., 2022; Gat et al., 2024; Holderrieth and Erives, 2026; Nie et al., 2025; Zhu et al., 2025; Ye et al., 2025; Bie et al., 2025) decode multiple tokens in parallel at each forward pass, offering substantially faster generation than autoregressive (AR) models (Song et al., 2025; DiffusionGemma Team, 2026; Wang et al., 2026c).

Early DLMs denoised the whole output within a single canvas with bidirectional attention (Austin et al., 2021; Lou et al., 2024). Since the canvas length is fixed, every forward pass contains all positions regardless of how many tokens need updating, and generation cannot extend beyond the canvas boundary. Block diffusion (Arriola et al., 2025) addresses this by splitting the canvas into blocks decoded sequentially: each forward pass refines only the current block, new blocks can be appended as needed, and each block attends causally to the previous blocks via a KV cache. This approach is widely used in recent large-scale DLMs (Bie et al., 2025; Wu et al., 2026a; Cheng et al., 2025; Song et al., 2025; Wang et al., 2026c). State-of-the-art block DLMs (DiffusionGemma Team, 2026; Bie et al., 2026) learn to self-edit from uniform-state corruption (Austin et al., 2021), and may spend further forward passes refining a block before moving

[1]Technion – Israel Institute of Technology. Work done during an internship at Amazon. [2]Amazon Web Services. Correspondence to Jonathan Fhima, fhimajon@amazon.com.

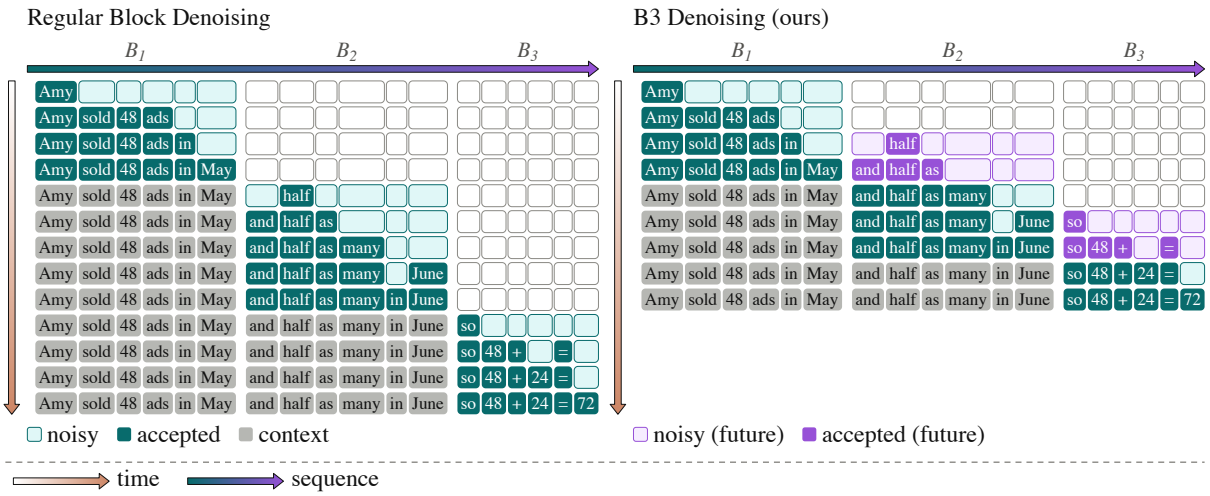

Figure 2: Schematic of *B3*. Each row is one forward pass. Left: vanilla decoding denoises one block at a time. Right: *B3* starts the next block early, sharing forward passes with the current block, and finishes in fewer passes.

on to the next.

A key observation of ours is that this artificial split into blocks carries an inherent inefficiency at block boundaries. Tokens are not equally easy to predict: some settle in a step or two, others need many more, and a block is not finalized until its slowest token is accepted. A forward pass over a block has the same computational cost regardless of how many tokens are accepted. At the end of a block's denoising process, the model accepts its last few tokens or self-edits earlier ones, while the next block's easy to predict tokens wait behind them, untouched. We show empirically that the number of tokens accepted per forward pass drops sharply near the block boundaries (Figure 1(a)), consistently across models and datasets (Appendix C).

In this work we present **Blurring the Block Boundaries (*B3*)**, a novel approach that reduces this loss in token throughput by blurring the artificial block boundaries. Once the current block is settled enough, the next block is drafted in a shared forward pass, conditioned on the current block's evolving state, parallelizing the end of the current block generation and the beginning of the next one (Figure 1(b)). Attention stays block-causal, so the current block's predictions are left unchanged. Once the draft becomes the current block, self-editing corrects any errors it carries from being initialised before the previous block had fully converged. The spawning time of the next block is dynamically determined from statistics of the predicted tokens in previous blocks, making *B3* training-free and hyperparameter-free, thus requiring no adaptation to integrate into existing models.

Evaluated on DiffusionGemma 26B (DiffusionGemma Team, 2026), LLaDA 2.1 mini 16B, and LLaDA 2.1 flash 100B (Bie et al., 2026) across 10 diverse datasets, *B3* delivers 11.5–14.5% speedup at on par generation quality without any training or hyperparameter tuning, advancing the latency–quality Pareto frontier of DLM inference.

To summarise, our key contributions are:

- Identifying an inefficiency of block diffusion, where the number of tokens accepted per forward pass drops near the block boundaries.
- Introducing *B3*, a novel approach which denoises two consecutive blocks in shared forward passes.
- Designing a self-calibrating spawning criterion that requires no training or hyperparameter tuning.
- Extensive evaluation of *B3* on three self-editing state-of-the-art block DLMs and 10 benchmarks.

2 Related Work

Diffusion language models. Diffusion language models denoise either continuous token embeddings (Li et al., 2022; Gulrajani and Hashimoto, 2023; Chen et al., 2026) or discrete tokens. Discrete diffusion models are characterised by their corruption process, which determines whether generation can revise its own tokens. Masked diffusion replaces tokens with an absorbing `[MASK]` state (Sahoo et al., 2024; Ou et al., 2025; Nie et al.,

2025; Ye et al., 2025), so generation progressively unmasks positions while revealed tokens are not revisited. Uniform-state diffusion replaces them with random vocabulary tokens (Austin et al., 2021; Sahoo et al., 2025; Ye et al., 2026; DiffusionGemma Team, 2026), so every position is re-predicted at every step. Hybrid models mix both (von Rütte et al., 2025; Bie et al., 2026). Masked models can be extended with self-editing through remasking (Wang et al., 2025) or self-correction objectives (Schiff et al., 2026; Wang et al., 2026b). Block DLMs (Arriola et al., 2025), adapted from pretrained AR models (Wu et al., 2026a; Fu et al., 2026) or scaled to 100B parameters (Bie et al., 2025), now underlie most recent large-scale DLMs.

Accelerating DLM inference. Block diffusion (Arriola et al., 2025) decodes the sequence block by block, so the clean prefix is cached exactly rather than approximated as in full-canvas DLMs (Wu et al., 2026b; Ma et al., 2025; Hu et al., 2026; Liu et al., 2026; Jiang et al., 2026). Within each block, sampling strategies accept more tokens per step via confidence thresholds (Wu et al., 2026b; Bie et al., 2025), entropy bounds (Ben-Hamu et al., 2025) and adaptive parallel decoding (Israel et al., 2025).

Multi-block decoding. Orthogonally to these accelerations, multi-block methods change which blocks share a forward pass, and therefore run on top of the model's native sampler. For DLMs trained without blocks, sliding and adaptive schemes reshape the block imposed by the sampler at inference (Luo et al., 2026; Zhang et al., 2026; Lu et al., 2026; Wang et al., 2026a; Wan et al., 2026; Zou et al., 2026). Concurrently to our work, FlowBlock (Tian et al., 2026) and MBD-LMs (Jin et al., 2026) overlap consecutive blocks of block DLMs, advancing to the next block at a tuned progress threshold. In contrast, *B3* is hyperparameter-free and achieves a better latency–quality trade-off than both (Table 2). *B3* also applies to USDMs, to which others are not adapted.

3 Method

3.1 Preliminaries

Discrete diffusion language models generate a sequence $x = (x^1, \ldots, x^L)$ by learning to reverse a token-level corruption process. At noise level $t \in [0, 1]$, from clean ($t = 0$) to fully corrupted ($t = 1$), each position $\ell \in \{1, \ldots, L\}$ of the noisy

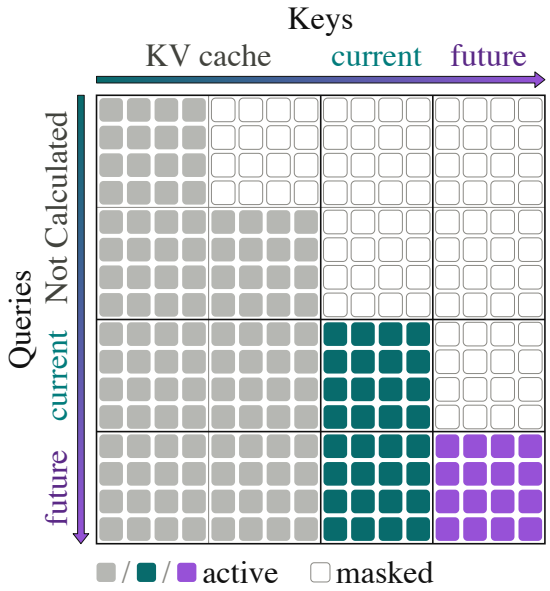

Figure 3: *B3* attention mask of a shared forward pass; the current block never attends to the future block.

sequence x_t independently either keeps token x_t^ℓ or is replaced by a token drawn from a corruption distribution π, with the replacement probability increasing in t. A denoiser $p_\theta(x^\ell \mid x_t)$ predicts the clean token at every position ℓ. For MDMs, π always yields a special `[MASK]` token. For USDMs, π is uniform over the vocabulary, so any token may be corrupted and p_θ learns to re-predict every position, enabling self-editing. Hybrid models (Bie et al., 2026; von Rütte et al., 2025) mix the two corruption distributions at train time, and so inherit the self-editing capabilities of the USDM component.

At inference, every position is initialised from π and p_θ is applied iteratively. Each denoising step is one forward pass of p_θ that predicts tokens at all L positions. A sampler selects a subset of these predicted tokens to retain in the sequence for the next denoising step by using a confidence threshold (Wu et al., 2026b; Bie et al., 2025) or an entropy bound (Ben-Hamu et al., 2025). Retained tokens accumulate over denoising steps until the whole sequence is complete.

Block diffusion models (Arriola et al., 2025) split the sequence into bidirectional blocks and generate them causally. We write x^b for the tokens in the b-th block $B_b \subset \{1, \ldots, L\}$, $x^{<b}$ for tokens in all preceding blocks, and $\ell \in B_b$ for a position ℓ in block b. The sampler runs only within the current block b, and each step denoises the block's noisy state $x_{t_b}^b$ at noise level t_b, conditioned on the

clean prefix:

$$p_\theta\big(x^b \mid x^{<b}, x^b_{t_b}\big) \tag{1}$$

Once x^b meets the sampler's convergence criterion, x^b is finalized and moved into the context, and block $b+1$ is initialised from π, unless an EOS has been generated in x^b. Attention is block-causal: a position $\ell \in B_b$ attends only to blocks $1, \ldots, b$, so each forward pass processes only block b, reading $x^{<b}$ from an exact KV cache.

3.2 Blurring the Block Boundaries

As block diffusion turns every block boundary into a synchronisation barrier, no position in block $b+1$ may start until the entire block b has been finalized. Empirical analysis (Section 4.5) identifies a significant efficiency drop in token generation around those artificial boundaries.

Rather than waiting for the current block to be finalized before initialising the next one, *B3* starts denoising the future block simultaneously with the current block (Figure 2) when approaching the boundaries: once the current block b is *settled enough* according to our proposed criterion (Section 3.3), the future block $b+1$ is initialised and both blocks are denoised together until b is finalized. Concretely, both blocks share a single forward pass through p_θ:

$$p_\theta\big(x^b, x^{b+1} \mid x^{<b}, x^b_{t_b}, x^{b+1}_{t_{b+1}}\big) \tag{2}$$

Because attention is block-causal (Figure 3), no position in block b attends to $x^{b+1}_{t_{b+1}}$, so the predictions for x^b are identical to those of the vanilla forward pass of eq. (1).

Each block keeps its own sampler, with separate noise levels t_b and t_{b+1}, so block b is denoised exactly as in standard block decoding. The future block, however, is conditioned on $x^b_{t_b}$, which may still contain tokens sampled from π, that can mislead its early predictions. Self-editing lets the sampler of block $b+1$ revise already accepted tokens of x^{b+1} after x^b converges. Once x^b is finalized, block $b+1$ becomes the current block and continues its denoising conditioned on the clean prefix $x^{<b+1}$.

3.3 Future Block Spawning

During training, different blocks are denoised disjointly. On the one hand, waiting for the current block to fully converge at inference time spawns the next block too late and prevents the desired overlap. On the other hand, spawning the future block too early conditions it on a still evolving current block, which may cause inference to deviate too far from training. Hence, the future block should be spawned once the tokens of the current block are settled enough.

The model entropy is a well-established (Ben-Hamu et al., 2025; Zhang et al., 2026) measure of how settled a token is,

$$H^\ell_i = -\sum_{v \in \mathcal{V}} p_\theta\big(x^\ell = v \mid x^{<b}, x^b_{t_b}\big) \cdot \log p_\theta\big(x^\ell = v \mid x^{<b}, x^b_{t_b}\big) \tag{3}$$

at position $\ell \in B_b$ on forward pass i of the denoising process at the corresponding noise level t_b, with the vocabulary set $\mathcal{V}$. The future block is then spawned once the entropy of every token in the current block b is at most a threshold τ,

$$\max_{\ell \in B_b} H^\ell_i \le \tau \tag{4}$$

For the method to remain hyperparameter-free, the remaining question is how to set τ without tuning it per model or task.

Intuitively, eq. (4) asks whether the tokens of B_b are settled enough for the model to start accepting tokens in B_{b+1}. We wish to estimate an entropy level sufficient for this decision from the model's own acceptances within B_b. Since B_{b+1} attends to B_b causally, we look for steps within B_b on which the generation advances approximately causally. Let $I_i \subseteq \{1, \ldots, L\}$ be the set of positions accepted over the generation up to forward pass i, accumulated across blocks.[1] We call its rightmost position, $r_i = \max I_i$, the generation frontier. Whenever the frontier advances, $r_i > r_{i-1}$, the newly accepted token at r_i has, like the first token of B_{b+1}, all of its accepted context to its left. The maximum entropy H^ℓ_{i-1} of the tokens at positions $\{\ell \mid \ell < r_i\}$ on the previous pass is therefore a candidate for τ that was sufficient for advancing the frontier to r_i,

$$\tau^{\text{candidate}}_i = \max_{\ell \in B_b,\ \ell < r_i} H^\ell_{i-1} \tag{5}$$

where r_i itself is excluded, just as the first token of B_{b+1} is excluded from eq. (4). The positions $\{\ell \mid \ell < r_i\}$ and r_i in eq. (5) thus play the roles of B_b and B_{b+1} in eq. (4). The one-step lag is also the same, since the gate evaluates the entropies of

[1] For DiffusionGemma, see Appendix G.

Model	Benchmark	n	TPS			TPF			Quality		
			Base	*B3* (ours)	Δ%	Base	*B3* (ours)	Δ%	Base	*B3* (ours)	Δ*pp*
DiffusionGemma 26B	GSM8K	1319	136.1	150.8	+10.8	28.74	33.69	+17.2	95.75	96.13	+0.38
	MATH500	500	139.9	156.0	+11.6	28.69	34.16	+19.1	95.80	92.40	-3.40
	AIME26	30	101.0	121.6	+20.3	20.61	25.48	+23.6	73.33	76.67	+3.33
	Omni-MATH	4428	107.6	118.0	+9.6	21.70	25.10	+15.7	61.50	60.68	-0.81
	GPQA-D	198	94.4	107.1	+13.4	19.18	22.88	+19.3	73.74	71.72	-2.02
	IFEval	541	103.4	119.9	+16.0	21.08	26.61	+26.3	95.80	95.44	-0.36
	HumanEval+	164	148.9	160.7	+7.9	30.77	37.53	+22.0	91.46	89.63	-1.83
	MBPP+	378	136.4	149.6	+9.7	28.39	33.40	+17.6	80.69	82.28	+1.59
	BBEH	4520	116.2	126.6	+8.9	24.16	28.53	+18.1	36.72	37.01	+0.29
	LCB	342	108.0	119.2	+10.4	22.11	25.59	+15.7	70.47	69.30	-1.17
	Average		119.2	132.9	+11.5	24.54	29.30	+19.4	77.53	77.13	-0.40
LLaDA 2.1 flash 100B	GSM8K	1319	90.7	98.5	+8.5	5.58	6.37	+14.0	94.01	93.56	-0.45
	MATH500	500	106.6	123.1	+15.5	6.56	7.55	+15.1	95.20	94.60	-0.60
	AIME26	30	75.4	82.0	+8.7	4.71	5.21	+10.4	66.67	76.67	+10.00
	Omni-MATH	4428	77.2	84.9	+10.1	4.73	5.24	+10.8	59.94	59.49	-0.45
	GPQA-D	198	52.9	53.6	+1.2	3.25	3.43	+5.7	66.67	68.18	+1.52
	IFEval	541	26.2	27.8	+5.9	1.62	1.71	+5.9	88.85	89.21	+0.36
	HumanEval+	164	211.1	254.3	+20.4	13.20	16.40	+24.2	85.37	84.76	-0.61
	MBPP+	378	131.7	157.5	+19.6	8.16	9.90	+21.4	76.72	74.34	-2.38
	BBEH	4520	66.2	71.6	+8.2	4.07	4.48	+10.2	12.48	14.16	+1.67
	LCB	342	87.9	98.2	+11.7	5.49	6.21	+13.1	41.52	40.06	-1.46
	Average		92.6	105.2	+13.6	5.74	6.65	+15.9	68.74	69.50	+0.76
LLaDA 2.1 mini 16B	GSM8K	1319	114.2	124.8	+9.3	4.88	5.49	+12.5	90.90	89.84	-1.06
	MATH500	500	160.4	186.1	+16.0	6.85	8.14	+18.9	92.80	91.80	-1.00
	AIME26	30	113.7	125.9	+10.7	4.92	5.46	+11.0	46.67	50.00	+3.33
	Omni-MATH	4428	115.5	127.8	+10.7	4.91	5.55	+13.0	46.39	45.93	-0.45
	GPQA-D	198	69.5	73.8	+6.1	3.02	3.26	+8.0	52.53	52.02	-0.51
	IFEval	541	31.1	32.4	+4.2	1.35	1.40	+4.3	86.45	85.97	-0.48
	HumanEval+	164	247.2	301.4	+21.9	10.94	13.69	+25.2	83.54	81.71	-1.83
	MBPP+	378	163.6	194.2	+18.7	7.16	8.57	+19.6	69.84	69.05	-0.79
	BBEH	4520	69.4	73.5	+5.8	2.98	3.22	+7.8	6.91	5.94	-0.96
	LCB	342	113.7	131.8	+15.9	5.10	5.83	+14.3	25.73	26.61	+0.88
	Average		119.8	137.2	+14.5	5.21	6.06	+16.3	60.17	59.89	-0.28

Table 1: Per-benchmark results for *B3* on DiffusionGemma 26B, LLaDA 2.1 flash 100B, and LLaDA 2.1 mini 16B.

pass i to decide whether tokens of B_{b+1} may be accepted from pass $i+1$ on.

Candidates are collected over the generation at hand, on the forward passes on which the frontier advances. Passes with an active future block are excluded, as the frontier then lies in B_{b+1} rather than in B_b, giving

$C_\tau = \{i \mid r_i > r_{i-1},\ B_{b+1} \text{ is not active at } i\}$.

Averaging the candidates of C_τ gives the threshold,

$$\tau = \frac{1}{|C_\tau|} \sum_{i \in C_\tau} \tau_i^{\text{candidate}}. \tag{6}$$

Finally, to avoid spawning future blocks past the end of the sequence, spawning is also gated on the current block not predicting an EOS.

4 Experiments

Models. We evaluate *B3* on DiffusionGemma 26B (DiffusionGemma Team, 2026), LLaDA 2.1 flash 100B and LLaDA 2.1 mini 16B (Bie et al., 2026), using their public checkpoints and Hugging Face implementations. DiffusionGemma uses its shipped decoding defaults. For both LLaDA variants we use block size 64 instead of the default 32, as it is faster at comparable accuracy (Figure 5). This choice favors the baseline, as it lowers the relative speedup of *B3* from 20% to 15%. Hardware and other technical details in Appendix A.

Metrics. We report Tokens per Second (TPS) calculated as output tokens divided by generation time in seconds, and Tokens per Forward (TPF) calculated as output tokens divided by model forward passes. We compute TPS and TPF for every input sample and report the mean over the benchmark. Δ% compares these means, $(T_{method} - T_{base})/T_{base}$. In the Average row, we first average TPS (or TPF) over benchmarks and then take Δ%. For a measure of quality we report Accu-

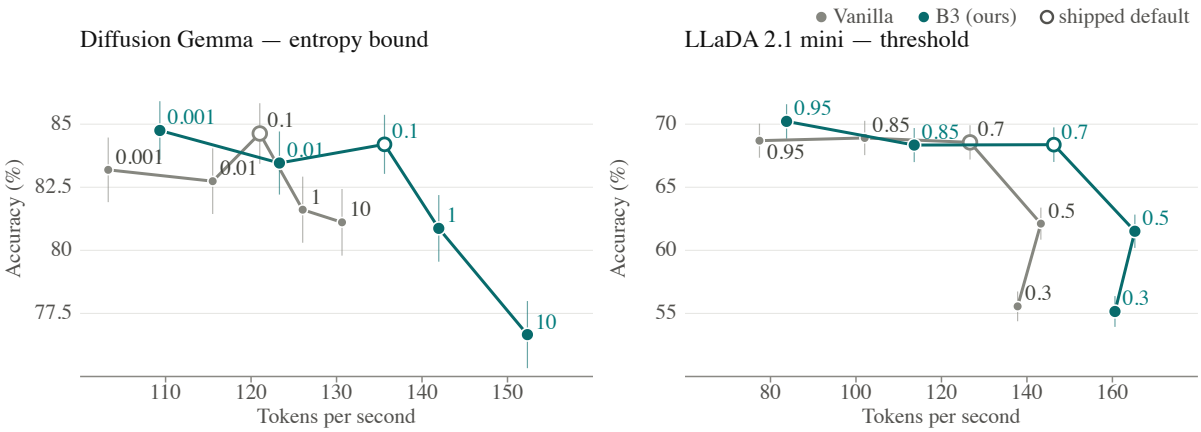

Figure 4: Sweep of the acceptance criterion. Error bars denote ±1 standard error of the average accuracy due to question sampling, combined from per-benchmark binomial standard errors.

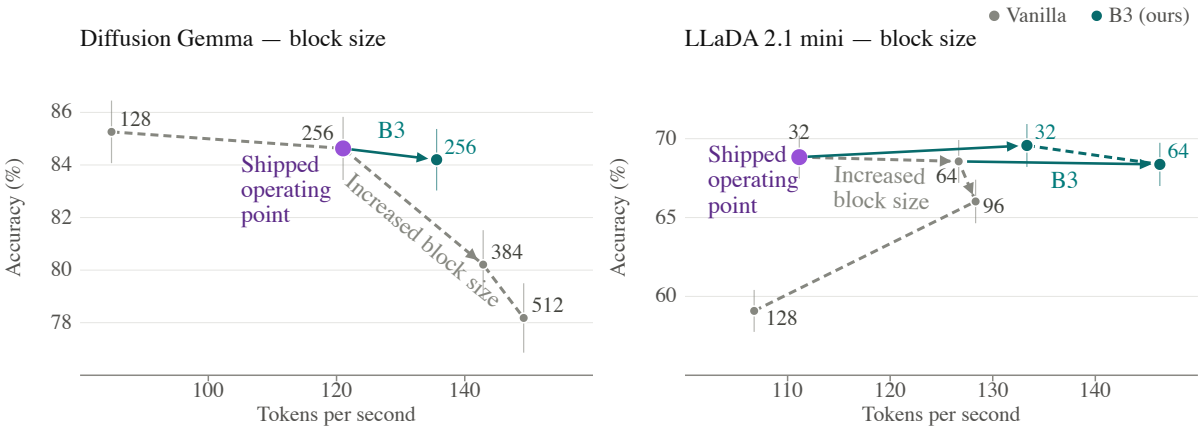

Figure 5: *B3* shows a preferable accuracy/speed trade-off compared to increasing the block size. At each shipped operating point, it is better to use *B3* instead of increasing the block size.

racy, defined as is common per dataset, and for ease of comparison we report also Δpp measuring the difference in absolute performance (accuracy is already scaled as a unitless fraction) $\Delta pp = A_{method} - A_{base}$. For all relative metrics we mark in teal improvements over the baseline and in vermilion degradation over it.

Datasets. We use the full test sets of 10 benchmarks, zero-shot except for 8-shot GSM8K (Using `gsm8k_cot` of `lm-evaluation-harness`; Gao et al., 2024), all with their common metrics.

- Math: GSM8K (Cobbe et al., 2021), MATH500 (Hendrycks et al., 2021; Lightman et al., 2024), AIME 2026 (Balunović et al., 2025) and Omni-MATH (Gao et al., 2025), scored by exact match of the final answer, checked for mathematical equivalence with Hugging Face Math-Verify[2] on MATH500 and Omni-MATH.
- Science: GPQA Diamond (Rein et al., 2024).
- Reasoning: BBEH (Kazemi et al., 2025), scored by adjusted harmonic mean of task accuracy.
- Instruction following: IFEval (Zhou et al., 2023), loose instruction-level accuracy.
- Code: HumanEval+ and MBPP+ (Liu et al., 2023) and LiveCodeBench (Jain et al., 2025), scored as pass@1.

Since *B3* is hyperparameter-free, no validation sets are used for tuning. Due to compute constraints, ablations and multi-block comparisons (Sections 4.2 and 4.3) exclude Omni-MATH and BBEH (4428 and 4520 examples) and report averages over the remaining eight, referred to as the *light* benchmarks.

4.1 Results

Table 1 reports results on DiffusionGemma, LLaDA 2.1 flash and LLaDA 2.1 mini, spanning

[2] `https://github.com/huggingface/Math-Verify`

Block Size	Method	TPS	TPF	Acc
32	Base	111	4.83	68.8
	FlowBlock	132 (+18%)	5.77 (+20%)	67.9 (-1.0)
	MBD-LMs	131 (+18%)	**5.95** (+23%)	67.4 (-1.4)
	B3	**133** (+20%)	5.88 (+22%)	**69.6** (+0.7)
64	Base	127	5.53	68.6
	FlowBlock	132 (+4%)	5.89 (+7%)	68.3 (-0.3)
	MBD-LMs	142 (+12%)	6.40 (+16%)	65.3 (-3.2)
	B3	**146** (+15%)	**6.48** (+17%)	**68.4** (-0.2)

Table 2: LLaDA 2.1 mini 16B: *B3* vs. concurrent inter-block methods on the eight light benchmarks described in Section 4.2.

two model families and 16B to 100B parameters. Averaged over benchmarks, *B3* raises TPS by 11.5–14.5% and TPF by 15.9–19.4%, while average accuracy changes only by –0.40 to +0.76 pp. The acceleration is consistent: *B3* improves both TPS and TPF on every one of the 30 model–benchmark pairs, without any per-model or per-task tuning.

4.2 Comparison with Other Multi-Block Methods

Table 2 compares *B3* with FlowBlock and MBD-LMs (training-free variant) on LLaDA 2.1 mini, using each method's published hyperparameters on a per-dataset basis.[3] Averaged over the eight light benchmarks at LLaDA 2.1's default block size of 32, *B3* is the fastest method, raising TPS by 2% more than both baselines, while being 1.7 pp and 2.1 pp more accurate than FlowBlock and MBD-LMs, respectively. At the faster block size of 64, *B3* again leads on both axes: it is 11% faster than FlowBlock at comparable accuracy, and 3.0 pp more accurate than MBD-LMs at 3% higher TPS. Per-dataset results are in Table 7.

4.3 Latency–Quality Trade-off

Two axes control the latency–quality trade-off in block diffusion models: the sampler acceptance criterion and the block size. We ablate both with and without *B3*, using each baseline's native parameters (entropy bound for DiffusionGemma and confidence threshold for LLaDA 2.1) as the control variable. LLaDA 2.1 is evaluated using the mini model rather than flash due to compute constraints; full per-benchmark results are shown in Tables 5 and 6.

[3]The baselines' math hyperparameters are used for math tasks and GPQA-D, and their code hyperparameters for code tasks and IFEval. Where the released code differs from the paper, the code is followed.

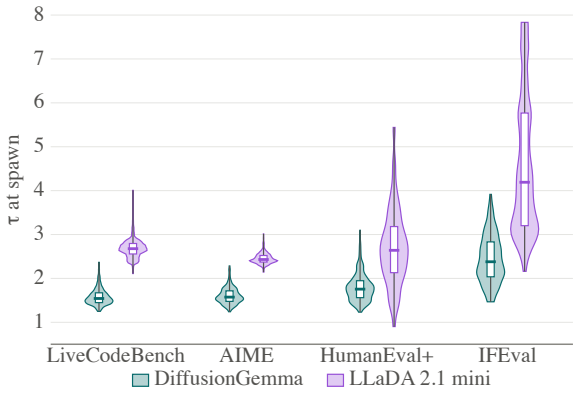

Figure 6: τ selected by the self-calibrating gate at each spawn, per benchmark and model. Violin: P1–P99; box: P25–P75; line: median.

Sampler threshold. Figure 4 sweeps the acceptance criterion with and without *B3*. In the high-quality regime, *B3* raises TPS at comparable quality, shifting the frontier, for example gaining +11% TPS at –0.2 pp for LLaDA 2.1 mini at threshold 0.95. At the shipped defaults, *B3* is faster than every baseline configuration in the sweep by 3.8% over the fastest DiffusionGemma baseline, which is 3.1 pp less accurate, and by 4.3% over the fastest LLaDA baseline, which is 10.3 pp less accurate.

Block size. Figure 5 varies the block size with and without *B3*. For DiffusionGemma, *B3* raises TPS by 12% at block size 256, a speed the baseline matches only at 384 or larger, at a cost of at least 4.0 pp quality, while *B3* itself loses only 0.4 pp. For LLaDA 2.1 mini, blocks beyond 64 give negligible speedup and lose quality, while *B3* raises TPS by 20% at block size 32 and 15% at 64 with comparable quality, the latter being faster than every baseline configuration. Enlarging the block therefore does not recover the speedup of *B3* at comparable quality.

4.4 Ablation on Future Block Spawning Criteria

Figures 7 and 8 ablate the spawn gate on HumanEval+. Spawning once a fixed fraction θ_{spawn} of the current block is accepted does not match the accuracy of our gate on either model.

A fixed entropy threshold τ is no easier to set. The τ selected by our gate varies across benchmarks and models (Figure 6; see Appendix D for a breakdown by dataset and block index). Accuracy is also sensitive to the fixed value chosen (Figure 7): $\tau = 1$ beats our gate on LLaDA 2.1 mini but is the

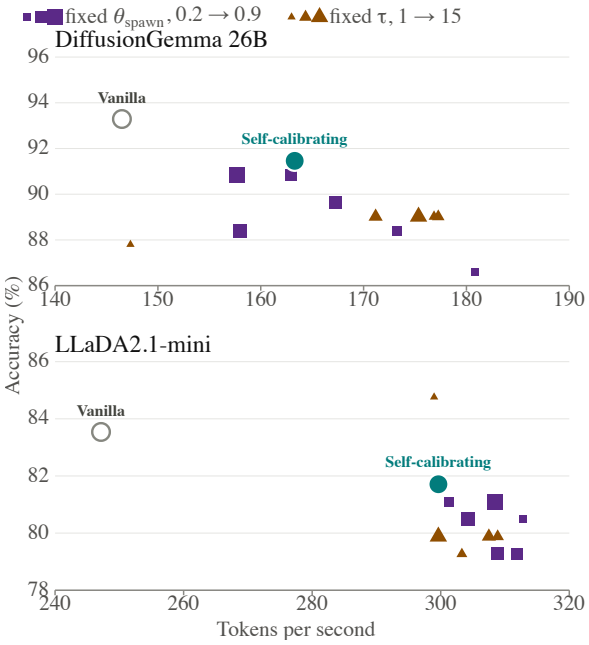

Figure 7: Ablation on HumanEval+ for our self-calibrating spawning threshold vs. spawning at a fixed fraction of acceptance, θ_{spawn}, and at a fixed max entropy gate of τ. Top right is preferable.

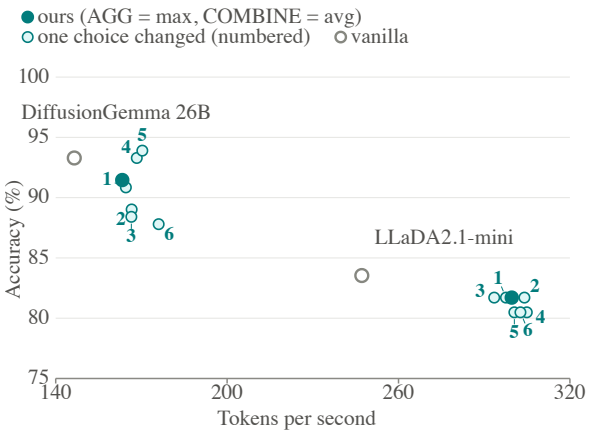

Figure 8: Design choices of the spawn gate on HumanEval+. Each numbered point changes one choice of ours (AGG = max, COMBINE = avg, C_τ carried across blocks): (1) C_τ resets every block, (2) AGG = avg, (3) AGG = sum, (4) COMBINE = avg + std, (5) COMBINE = max, (6) COMBINE = max (corrected). AGG and COMBINE are defined in Section 4.4.

least accurate fixed setting on DiffusionGemma. A fixed τ would therefore need tuning per model and dataset, which *B3* avoids.

Finally, Figure 8 changes one design choice of the gate at a time. This includes how the entropies left of r_i are reduced to a candidate (AGG; ours: max, eqs. (4) and (5)) and how the candidates are combined into τ (COMBINE; ours: mean, eq. (6)). All variants land close to ours in speed and accuracy, and none improves on it consistently across both models.

4.5 Inefficiency at the Block Boundary

Figure 1 illustrates the slow finish of a block under vanilla decoding: acceptance falls as the block nears completion. For DiffusionGemma on HumanEval+, the last 20% of a block takes 22% of the steps but yields only 10% of the tokens (first and last block excluded). This holds for both models on HumanEval+ and GPQA Diamond. The ratio of token share to step share in the last 20% (1 being parity) is 0.35–0.71 for DiffusionGemma and 0.05–0.62 for LLaDA 2.1 mini (Figure 11). With *B3*, the next block accepts tokens on the same forward passes, raising the ratio to 0.67–1.39 and 0.71–1.25. Appendix C breaks this down by block index for DiffusionGemma.

5 Conclusion

We introduced *B3*, a training-free and without hyperparameters acceleration method for self-editing block diffusion language models. *B3* exploits a structural property of these models: late denoising steps of a block accept few tokens, so the next block can start early on the same forward passes. An entropy-based gate calibrates the spawning point from the model's own acceptance behaviour.

Experiments on three models spanning two diffusion families, hybrid (LLaDA 2.1 mini 16B and flash 100B) and USDM (DiffusionGemma 26B), and 10 diverse benchmarks show that *B3* consistently improves TPS by 11.5–14.5% and TPF by 15.9–19.4%, with an average accuracy change of +0.03 pp over the three models, without any per-model or per-task tuning. On LLaDA 2.1 mini, *B3*, as a plug-and-play addition, also achieves a better latency–quality trade-off than concurrent multi-block methods with tuned thresholds.

Spawning the next block too late forfeits the speedup, while spawning it too early conditions it on a still evolving prefix that can mislead its predictions. The TPS gains suggest that the gate opens early enough to exploit the low-acceptance tail of each block. The preserved accuracy suggests that it also opens late enough for the settled tokens of the current block to provide reliable context for the next one. If any errors do slip past the gate, self-editing appears to correct them.

Limitations

B3 relies on self-editing for having low-acceptance forwards to exploit, as well as to correct future block tokens drafted on an unfinished prefix, so masked DLMs without an editing mechanism fall outside its scope. The state-of-the-art models we evaluate, in both diffusion families, self-edit by design. The spawning gate also assumes semi left-to-right generation at parts of the generation, which both evaluated models exhibit (Appendix E).

Unlike verification-based speculative decoding, *B3* does not guarantee outputs identical to vanilla decoding, although average accuracy changes by only -0.40 to $+0.76$ pp. Its speedup is bounded by the low-acceptance tail of each block. The resulting 11.5–14.5% TPS gain is moderate.

Wall-clock speedups are measured on one inference stack per model and may differ in other serving regimes, such as large-batch inference. Due to compute constraints, ablations and comparisons with concurrent methods use LLaDA 2.1 mini and a subset of benchmarks. The evaluation focuses on English only text only tasks.

References

Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*.

Ali Asaria, Tony Salomone, and Deep Gandhi. 2026. Neither parallel nor sequential: How DiffusionGemma actually commits tokens. *Preprint*, arXiv:2606.14620.

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*.

Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. 2025. MathArena: Evaluating LLMs on uncontaminated math competitions. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. 2025. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *Advances in Neural Information Processing Systems*.

Tiwei Bie et al. 2025. LLaDA2.0: Scaling up diffusion language models to 100B. *Preprint*, arXiv:2512.15745.

Tiwei Bie et al. 2026. LLaDA2.1: Speeding up text diffusion via token editing. *Preprint*, arXiv:2602.08676.

Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. 2022. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems*, volume 35, pages 28266–28279.

Yuxin Chen, Chumeng Liang, Hangke Sui, Ruihan Guo, Chaoran Cheng, Jiaxuan You, and Ge Liu. 2026. LangFlow: Continuous diffusion rivals discrete in language modeling. *arXiv preprint arXiv:2604.11748*.

Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. 2025. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

DiffusionGemma Team. 2026. DiffusionGemma technical report. *Preprint*, arXiv:2608.00146.

Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. 2026. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. In *International Conference on Machine Learning*.

Bofei Gao et al. 2025. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. In *International Conference on Learning Representations*.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. 2024. The language model evaluation harness. Version v0.4.3.

Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. 2024. Discrete flow matching. In *Advances in Neural Information Processing Systems*, volume 37, pages 133345–133385.

Ishaan Gulrajani and Tatsunori B. Hashimoto. 2023. Likelihood-based diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*.

Peter Holderrieth and Ezra Erives. 2026. An introduction to flow matching and diffusion models. *Preprint*, arXiv:2506.02070. MIT 6.S184 course notes.

Kai Syun Hou and James Kwok. 2026. Rethinking the generation order of block diffusion language models. *Preprint*, arXiv:2607.24306.

Zhanqiu Hu, Jian Meng, Yash Akhauri, Mohamed S. Abdelfattah, Jae sun Seo, Zhiru Zhang, and Udit Gupta. 2026. FlashDLM: Accelerating diffusion language model inference via efficient KV caching and guided diffusion. In *International Conference on Learning Representations*.

Daniel Israel, Guy Van den Broeck, and Aditya Grover. 2025. Accelerating diffusion LLMs via adaptive parallel decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*. Spotlight.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*.

Yuchu Jiang, Yue Cai, Xiangzhong Luo, Jiale Fu, Jiarui Wang, Chonghan Liu, and Xu Yang. 2026. d^{2}Cache: Accelerating diffusion-based LLMs via dual adaptive caching. In *International Conference on Learning Representations*.

Yijie Jin, Jiajun Xu, Yuxuan Liu, Chenkai Xu, Yi Tu, Jiajun Li, Dandan Tu, Xiaohui Yan, Kai Yu, Pengfei Liu, and Zhijie Deng. 2026. Multi-block diffusion language models. *Preprint*, arXiv:2606.29215.

Mehran Kazemi et al. 2025. BIG-Bench extra hard. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.

Maurice G. Kendall. 1938. A new measure of rank correlation. *Biometrika*, 30(1-2):81–93.

Maurice G. Kendall. 1945. The treatment of ties in ranking problems. *Biometrika*, 33(3):239–251.

Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham Kakade, and Sitan Chen. 2025. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *International Conference on Machine Learning*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, pages 611–626.

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. 2022. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*.

Zhiyuan Liu, Yicun Yang, Yaojie Zhang, Junjie Chen, Chang Zou, Qingyan Wei, Shaobo Wang, Yichen Zhu, and Linfeng Zhang. 2026. dLLM-Cache: Accelerating diffusion large language models with adaptive caching. In *International Conference on Machine Learning*.

Aaron Lou, Chenlin Meng, and Stefano Ermon. 2024. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning (ICML)*.

Guanxi Lu, Hao Mark Chen, Yuto Karashima, Zhican Wang, Daichi Fujiki, and Hongxiang Fan. 2026. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. In *International Conference on Learning Representations*.

Lizhuo Luo, Shenggui Li, Yonggang Wen, and Tianwei Zhang. 2026. DSB: Dynamic sliding block scheduling for diffusion LLMs. In *International Conference on Machine Learning*.

Xinyin Ma, Runpeng Yu, Gongfan Fang, and Xinchao Wang. 2025. dKV-Cache: The cache for diffusion language models. In *Advances in Neural Information Processing Systems*.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. In *Advances in Neural Information Processing Systems*.

Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. 2025. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *International Conference on Learning Representations*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A graduate-level google-proof Q&A benchmark. In *Conference on Language Modeling*.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*.

Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin Chiu, and Volodymyr Kuleshov. 2025. The diffusion duality. In *International Conference on Machine Learning*.

Yair Schiff, Omer Belhasin, Roy Uziel, Guanghan Wang, Marianne Arriola, Gilad Turok, Ran Zilberstein, Michael Elad, and Volodymyr Kuleshov. 2026. Learn from your mistakes: Self-correcting masked diffusion models. *Preprint*, arXiv:2602.11590.

Yuxuan Song et al. 2025. Seed diffusion: A large-scale diffusion language model with high-speed inference. *Preprint*, arXiv:2508.02193.

Bing Tian, Haikun Liu, Xiaocheng Zhong, Zhuohui Duan, Zhaokai Luo, Huayi Jin, Zhiyong Wang, and Xiaofei Liao. 2026. FlowBlock: Wavefront-parallel decoding for self-correcting diffusion language models. *Preprint*, arXiv:2607.17652.

Dimitri von Rütte, Janis Fluri, Yuhui Ding, Antonio Orvieto, Bernhard Schölkopf, and Thomas Hofmann. 2025. Generalized interpolating discrete diffusion. In *International Conference on Machine Learning*.

Lipeng Wan, Junjie Ma, Jianhui Gu, Zeyang Liu, Xuyang Lu, and Xuguang Lan. 2026. GeoBlock: Inferring block granularity from dependency geometry in diffusion language models. *Preprint*, arXiv:2603.26675.

Danny Wang, Ruihong Qiu, and Zi Huang. 2026a. When to commit? towards variable-size self-contained blocks for discrete diffusion language models. *Preprint*, arXiv:2604.23994.

Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. 2025. Remasking discrete diffusion models with inference-time scaling. In *Advances in Neural Information Processing Systems*.

Linxuan Wang, Ziyi Wang, Yikun Bai, Wei Deng, Guang Lin, and Qifan Song. 2026b. Generalized discrete diffusion with self-correction. In *International Conference on Machine Learning*.

Xu Wang, Chenkai Xu, Yijie Jin, Jiachun Jin, Hao Zhang, and Zhijie Deng. 2026c. Diffusion LLMs can do faster-than-AR inference via discrete diffusion forcing. In *International Conference on Learning Representations (ICLR)*.

Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. 2026a. Fast-dLLM v2: Efficient block-diffusion LLM. In *International Conference on Learning Representations*.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2026b. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. In *International Conference on Learning Representations*.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7B: Diffusion large language models. *Preprint*, arXiv:2508.15487.

Mengyu Ye, Keito Kudo, Wataru Ikeda, Ryosuke Matsuda, Keisuke Sakaguchi, and Jun Suzuki. 2026. Sumi: Open uniform diffusion language model from scratch. *Preprint*, arXiv:2606.19005.

Yu Zhang, Xinchen Li, Jialei Zhou, Hongnan Ma, Zhongwei Wan, Yiwei Shi, Duoqian Miao, Qi Zhang, and Longbing Cao. 2026. Swordsman: Entropy-driven adaptive block partition for efficient diffusion language models. In *International Conference on Machine Learning*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *Preprint*, arXiv:2311.07911.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025.

LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models. *Preprint*, arXiv:2505.19223.

Shun Zou, Yong Wang, Zehui Chen, Lin Chen, Chongyang Tao, Feng Zhao, and Xiangxiang Chu. 2026. Breaking block boundaries: Anchor-based history-stable decoding for diffusion large language models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.

Anthropic. 2026a. Claude. https://www.anthropic.com/claude. Accessed October 2026.

Anthropic. 2026b. Claude Code. https://www.anthropic.com/claude-code. Accessed October 2026.

A Hardware and Technical Implementation Details

Hardware. DiffusionGemma runs on 2 A100 40GB GPUs, LLaDA 2.1 flash on 2 H200 141GB GPUs, and LLaDA 2.1 mini on 1 A100 40GB GPU. Generation is capped only by each model's recommended context size: 32K total tokens for DiffusionGemma and 16K output tokens for LLaDA 2.1.

Inference implementation for LLaDA 2.1. We fix two inefficiencies in LLaDA 2.1's shipped Hugging Face inference code. First, we replace the MoE forward pass with the fused MoE kernel of vLLM (Kwon et al., 2023), as the public implementation uses a plain for loop to iterate experts, reducing GPU utilization to near 0 in our setup. We also implement KV caching of finalized blocks. This does not affect the mini model generation speed but without it runtimes for the flash checkpoint are quadratic in length. Both these changes were unrelated to our method, and we report all results (ours and baselines) over these changes.

B Per-Benchmark Results

We detail per-benchmark performance, for the tables and figures which present only averages in Tables 3 to 7.

C Block Boundary Inefficiency by Block Index and Dataset

Figures 9 and 10 break Figure 1 down by block index for DiffusionGemma on HumanEval+ and GPQA Diamond. Each row plots the tokens accepted per denoising step against the share of the block already accepted, for vanilla decoding (a, left) and for *B3* (b, right), on a scale shared by all rows. The shaded region is the block's last 20%: the steps taken while more than 80% of the block is accepted. Its label gives the share of vanilla's steps that fall there and the share of tokens those steps accept. At every block index, vanilla spends 19–24% of its steps there for 10–13% of the tokens. The last block of each generation is excluded, and the first row also excludes the first block, as in Figure 1.

Figure 11 summarises this statistic for both models: the share of a block's tokens accepted in its last 20%, divided by the share of its steps spent there, by the block's position in the generation. A value of 1 means the end of the block yields tokens in proportion to the steps it costs. Vanilla decoding stays below 1 throughout, at 0.35–0.71 for DiffusionGemma and 0.05–0.62 for LLaDA 2.1 mini. With *B3* the ratio is 0.67–1.39 and 0.71–1.25, since the next block accepts tokens on the same forward passes.

LLaDA 2.1 mini on HumanEval+ has the lowest ratio because LLaDA 2.1 forces editing steps once a block is fully accepted, and these accept no new tokens: 39% of its first block's steps run on a fully accepted block.

D Spawn Points of the Self-Calibrating Gate

Figures 12 to 16 show where the gate of Section 3.3 opens the future block, for DiffusionGemma and LLaDA 2.1 mini on HumanEval+ and GPQA Diamond. For every spawn, the top row gives the share of the current block already accepted, and the bottom row the threshold τ at that step. Gray bars count blocks that finished without spawning a successor. Spawns at the start of a generation, before τ has any candidate, have no τ and are left out of the bottom row.

The spawn point differs per dataset. DiffusionGemma spawns at a median of 84.0% of the block accepted on HumanEval+ and 89.5% on GPQA Diamond, at a median τ of 1.76 and 1.61. Across block indices its medians stay within 85–90%; blocks 5 and later come mostly from the longer GPQA Diamond generations. LLaDA 2.1 mini spawns close to the end of the block, at a median of 100% on HumanEval+ and 96.9% on GPQA Diamond.

E Frontier Advance within a Block

Section 3.3 treats each advance of the frontier r_i as a stand-in for spawning: in eq. (5), the token at

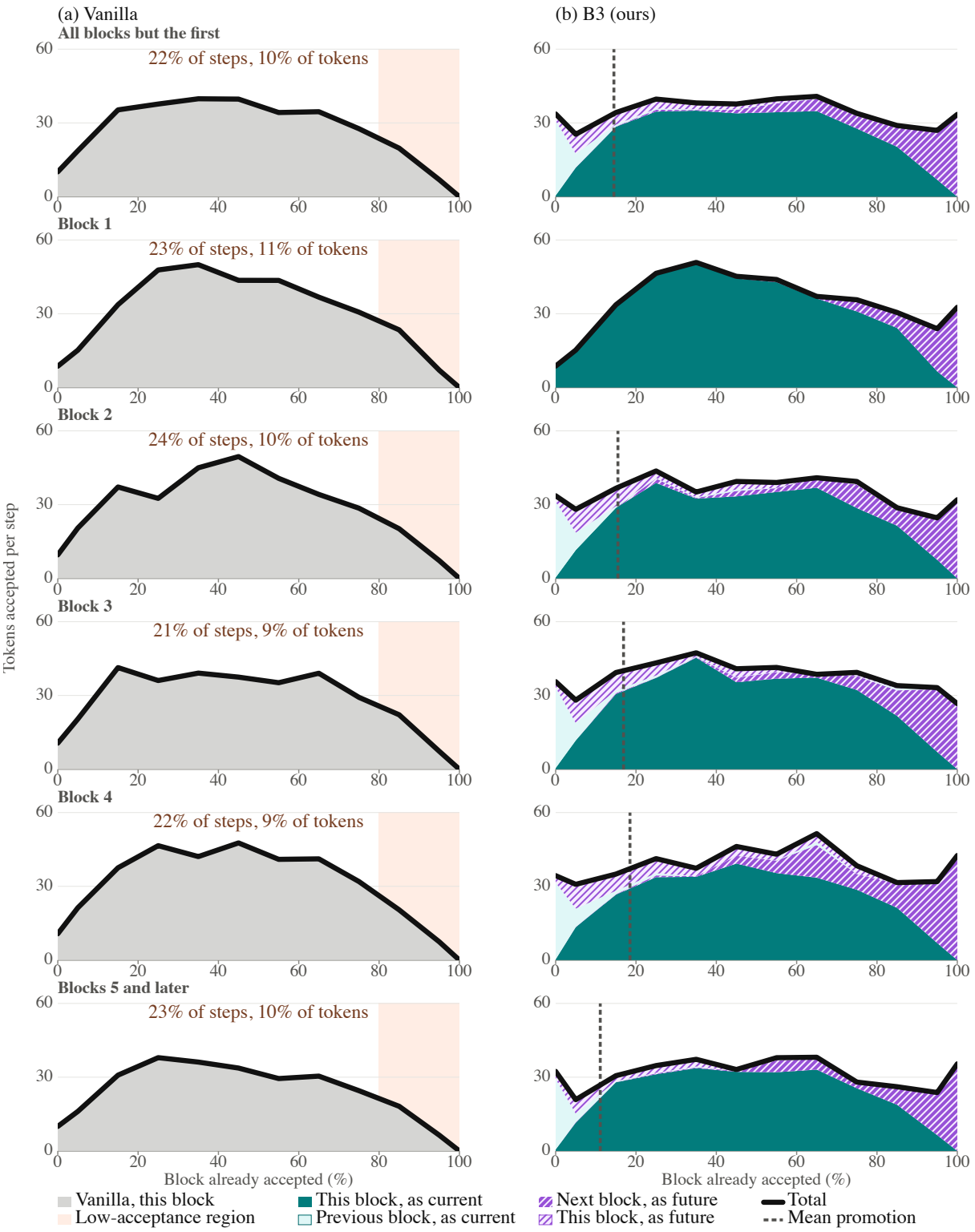

Figure 9: DiffusionGemma on HumanEval+: tokens accepted per step by block index. Vanilla left, *B3* right.

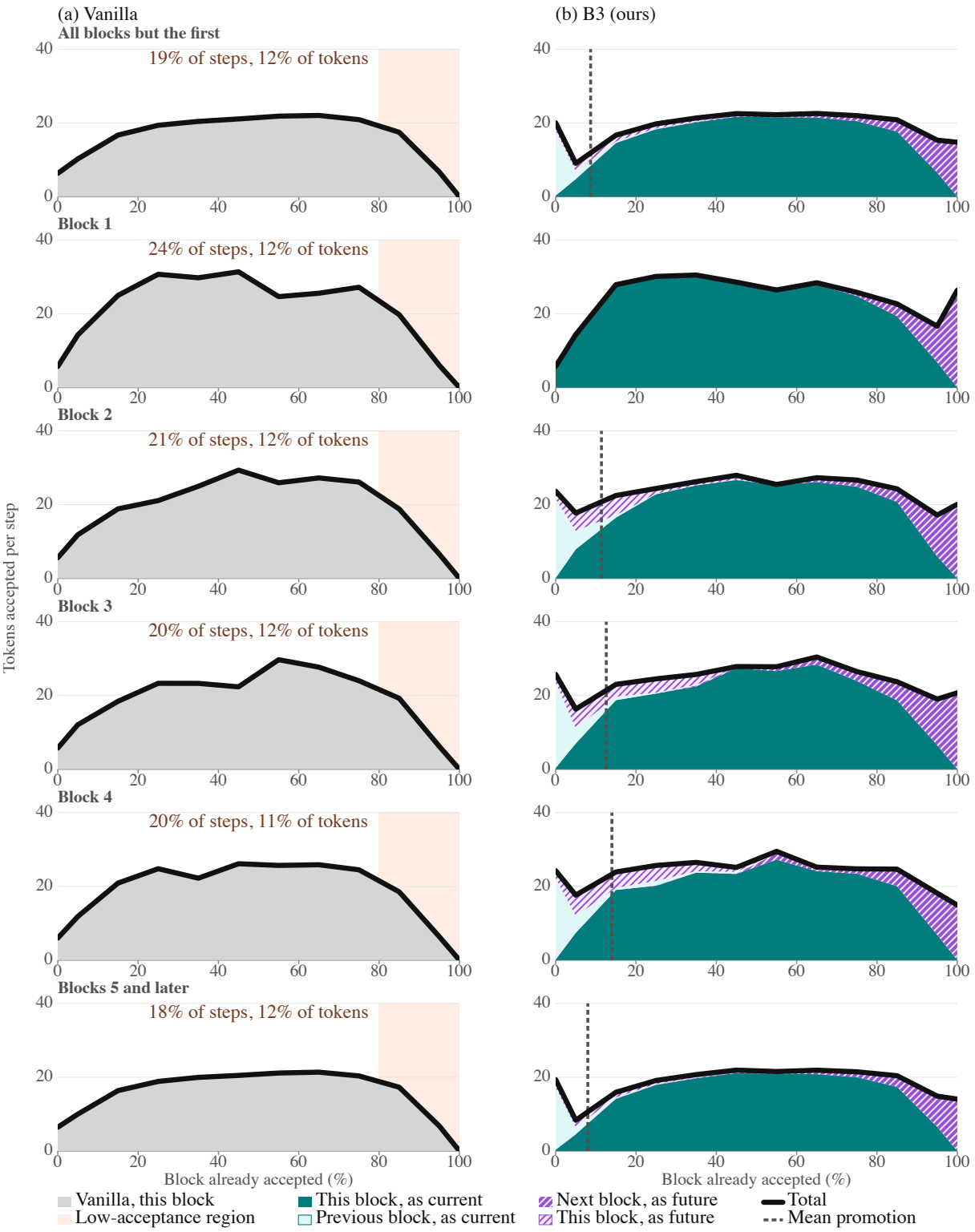

Figure 10: DiffusionGemma on GPQA Diamond: tokens accepted per step by block index. Vanilla left, *B3* right.

Entropy bound	0.001		0.01		0.1		1		10	
Method	Base	***B3***	Base	***B3***	Base	***B3***	Base	***B3***	Base	***B3***
TPS										
GSM8K	113.5	**119.2**	131.1	**141.0**	136.1	**150.8**	142.3	**160.6**	145.2	**167.7**
MATH500	125.2	**125.1**	137.3	**142.9**	139.9	**156.0**	144.0	**161.2**	147.0	**171.4**
AIME26	93.8	**97.7**	99.9	**108.2**	101.0	**121.6**	114.6	**120.0**	119.5	**129.6**
GPQA-D	76.8	**84.8**	86.3	**96.2**	94.4	**107.1**	99.4	**114.2**	103.6	**121.5**
IFEval	79.5	**92.3**	93.9	**108.8**	103.4	**119.9**	108.9	**128.8**	117.1	**139.7**
HumanEval+	128.5	**131.8**	141.9	**151.5**	148.9	**160.7**	143.6	**173.8**	157.6	**186.0**
MBPP+	121.2	**128.4**	130.1	**130.4**	136.4	**149.6**	142.6	**152.9**	139.0	**166.9**
LCB	87.9	**95.3**	103.7	**107.5**	108.0	**119.2**	113.0	**124.1**	116.2	**135.8**
Average	103.3	**109.3**	115.5	**123.3**	121.0	**135.6**	126.0	**142.0**	130.6	**152.3**
TPF										
GSM8K	23.69	**26.95**	27.11	**30.91**	28.74	**33.69**	30.04	**36.44**	31.07	**38.72**
MATH500	25.24	**27.37**	27.57	**31.25**	28.69	**34.16**	29.76	**36.69**	30.65	**39.65**
AIME26	18.38	**19.59**	21.11	**22.76**	20.61	**25.48**	22.92	**25.69**	23.86	**27.83**
GPQA-D	15.47	**17.82**	17.63	**20.42**	19.18	**22.88**	20.14	**24.66**	21.18	**27.78**
IFEval	16.14	**19.97**	19.22	**23.66**	21.08	**26.61**	22.47	**28.71**	24.11	**31.69**
HumanEval+	27.02	**29.92**	30.01	**34.39**	30.77	**37.53**	32.18	**40.59**	33.39	**43.65**
MBPP+	24.71	**27.48**	27.03	**30.16**	28.39	**33.40**	29.75	**35.79**	30.64	**38.20**
LCB	18.12	**20.04**	20.95	**23.24**	22.11	**25.59**	23.05	**27.90**	23.80	**30.46**
Average	21.10	**23.64**	23.83	**27.10**	24.95	**29.92**	26.29	**32.06**	27.34	**34.75**
Accuracy										
GSM8K	96.4	**96.4**	96.1	**95.5**	95.8	**96.1**	96.0	**95.8**	94.9	**95.0**
MATH500	96.8	**96.4**	95.8	**95.6**	95.8	**92.4**	95.0	**91.2**	94.8	**89.8**
AIME26	63.3	**76.7**	60.0	**66.7**	73.3	**76.7**	53.3	**56.7**	60.0	**40.0**
GPQA-D	69.7	**68.2**	71.2	**71.7**	73.7	**71.7**	70.2	**71.7**	74.2	**70.2**
IFEval	96.4	**96.4**	97.0	**96.4**	95.8	**95.4**	95.6	**95.1**	95.9	**93.8**
HumanEval+	92.1	**93.3**	90.2	**92.7**	91.5	**89.6**	91.5	**89.0**	84.1	**82.3**
MBPP+	80.4	**79.4**	82.3	**81.0**	80.7	**82.3**	81.7	**79.1**	79.6	**79.6**
LCB	70.5	**71.3**	69.3	**68.1**	70.5	**69.3**	69.6	**68.4**	65.2	**62.6**
Average	83.2	**84.8**	82.7	**83.5**	84.6	**84.2**	81.6	**80.9**	81.1	**76.7**

Table 3: DiffusionGemma 26B, per-benchmark results for Figure 4.

r_i plays the first token of B_{b+1} and the positions $\ell < r_i$ play B_b. This requires only two properties of the commitment order within a block:

(i) *Gradual advance.* r_i moves through the block in small steps. An early jump to the block's end would yield one candidate over a mostly unsettled block and then freeze r_i, so the block would contribute no further candidates.

(ii) *Left context drives the frontier.* The token at r_i is accepted mainly on the basis of the partly settled region to its left, as B_{b+1} is conditioned on B_b.

Commitment in positional order, one token per step, or with all of $\{\ell < r_i\}$ accepted is not required; eq. (5) takes the maximum over all positions left of r_i, as eq. (4) does over B_b. Self-editing behind the frontier is allowed.

Evidence in the literature. Confidence- and entropy-based samplers accept the most certain tokens first, inducing a task-dependent, easy-first order (Kim et al., 2025). On text, this order is commonly found to correlate with position. Sumi (Ye et al., 2026), a uniform-state DLM trained from scratch without blocks, shows a left-to-right commitment gradient on three of four tasks under confidence sampling, but not under ancestral sampling, so the gradient emerges at inference rather than being imposed by training. For DiffusionGemma, Asaria et al. (2026) report a partial left-to-right bias in commitment order on prose, code and math. Hou and Kwok (2026) find unmasking concentrated near the diagonal both for the block model SDAR and for the full-canvas LLaDA.

Measurements on the evaluated models. Figure 17 plots r_i against the share of the block committed after step i, for DiffusionGemma and LLaDA 2.1 mini under vanilla decoding on HumanEval+ and GPQA Diamond. An early jump of the frontier would lie far above the diagonal. LLaDA 2.1 mini stays within 3.4 points of it, and DiffusionGemma runs ahead by 6–8 points on average and at most 11.4, so in both models r_i ad-

Confidence threshold	0.3		0.5		0.7		0.85		0.95	
Method	Base	***B3***	Base	***B3***	Base	***B3***	Base	***B3***	Base	***B3***
TPS										
GSM8K	128.1	**146.1**	133.5	**150.1**	114.2	**124.8**	84.5	**91.9**	58.6	**61.5**
MATH500	160.6	**186.5**	171.2	**199.5**	160.4	**186.1**	126.5	**139.3**	95.2	**102.9**
AIME26	109.6	**115.8**	131.3	**149.9**	113.7	**125.9**	92.6	**92.9**	70.6	**72.1**
GPQA-D	65.6	**66.2**	79.8	**87.1**	69.5	**73.8**	54.4	**56.1**	42.8	**44.4**
IFEval	46.5	**49.5**	37.6	**40.0**	31.1	**32.4**	26.8	**27.8**	23.8	**24.0**
HumanEval+	271.7	**345.3**	271.3	**327.6**	247.2	**301.4**	213.9	**251.5**	168.5	**192.7**
MBPP+	184.2	**218.0**	183.7	**214.9**	163.6	**194.2**	124.4	**148.5**	91.2	**99.3**
LCB	136.2	**157.4**	137.8	**153.0**	113.7	**131.8**	93.2	**101.0**	68.6	**73.3**
Average	137.8	**160.6**	143.3	**165.3**	126.7	**146.3**	102.0	**113.6**	77.4	**83.8**
TPF										
GSM8K	5.61	**6.42**	5.74	**6.58**	4.88	**5.49**	3.65	**3.98**	2.55	**2.69**
MATH500	6.90	**8.27**	7.36	**8.78**	6.85	**8.14**	5.41	**6.22**	4.08	**4.59**
AIME26	4.62	**5.04**	5.68	**6.59**	4.92	**5.46**	3.88	**4.12**	3.05	**3.12**
GPQA-D	2.84	**2.90**	3.46	**3.85**	3.02	**3.26**	2.36	**2.49**	1.86	**1.93**
IFEval	2.03	**2.21**	1.66	**1.77**	1.35	**1.40**	1.15	**1.19**	1.02	**1.04**
HumanEval+	12.36	**15.60**	12.01	**14.97**	10.94	**13.69**	9.32	**11.38**	7.40	**8.85**
MBPP+	7.88	**9.63**	8.04	**9.52**	7.16	**8.57**	5.55	**6.54**	3.96	**4.43**
LCB	5.96	**7.02**	6.09	**6.83**	5.10	**5.83**	4.01	**4.47**	3.03	**3.24**
Average	6.02	**7.14**	6.26	**7.36**	5.53	**6.48**	4.42	**5.05**	3.37	**3.74**
Accuracy										
GSM8K	81.6	**79.9**	88.6	**87.6**	90.9	**89.8**	91.5	**91.1**	91.3	**91.1**
MATH500	85.8	**84.8**	89.6	**87.6**	92.8	**91.8**	91.2	**93.2**	92.6	**93.2**
AIME26	16.7	**20.0**	26.7	**30.0**	46.7	**50.0**	46.7	**40.0**	43.3	**53.3**
GPQA-D	37.4	**34.3**	40.9	**46.5**	52.5	**52.0**	51.0	**50.5**	47.5	**49.0**
IFEval	84.1	**83.1**	86.6	**85.6**	86.5	**86.0**	87.1	**87.3**	86.7	**86.9**
HumanEval+	64.6	**67.1**	78.0	**72.0**	83.5	**81.7**	85.4	**83.5**	85.4	**84.8**
MBPP+	57.4	**57.7**	66.4	**62.4**	69.8	**69.0**	70.4	**73.0**	73.5	**73.0**
LCB	17.0	**14.3**	20.2	**20.5**	25.7	**26.6**	28.1	**28.1**	29.2	**30.4**
Average	55.6	**55.2**	62.1	**61.5**	68.6	**68.4**	68.9	**68.3**	68.7	**70.2**

Table 4: LLaDA 2.1 mini 16B, per-benchmark results for fig. 4.

vances gradually, satisfying (i). The drop in the last bin comes from steps on an almost complete block, where the remaining acceptances can fall anywhere. The last block of each generation is excluded.

Figure 18 plots, for every position in a block, the step of its first acceptance as a share of the block's steps that accept at least one token. Per block, we measure the rank correlation between position and first-acceptance step with Kendall's τ_b (Kendall, 1938, 1945) (unrelated to the threshold τ), which handles ties from tokens accepted on the same step. Its median is 0.76 and 0.84 for DiffusionGemma on HumanEval+ and GPQA Diamond, and 0.62 and 0.90 for LLaDA 2.1 mini. It is positive in 99% of blocks in every case, yet clearly below 1 with a wide spread across positions: commitment is correlated with position without being sequential, consistent with (ii). Positions never accepted (0.4% of DiffusionGemma's) are left out.

F Self-Editing of Drafted Tokens

The future block is drafted while the current block is still being denoised, so it is conditioned on a prefix that may still hold noise (Section 3). Self-editing is what lets the sampler revise those early guesses once the prefix settles. Table 8 measures how often this happens. For every token of a block that is finalised, it compares the token accepted first with the token the block finalises with, on all 164 HumanEval+ problems. The vanilla rows count tokens accepted in the current block, the only kind vanilla decoding has; the *B3* rows count tokens accepted while their block was still the future block.

Tokens drafted in the future block are edited more often: 1.98% against 0.27% for DiffusionGemma, and 4.29% against 2.80% for LLaDA 2.1 mini. These are the small mistakes introduced by drafting on an unfinished previous block, and self-editing corrects them before the block is finalised. They stay few: 98.0% and 95.7% of the drafted tokens are kept unchanged.

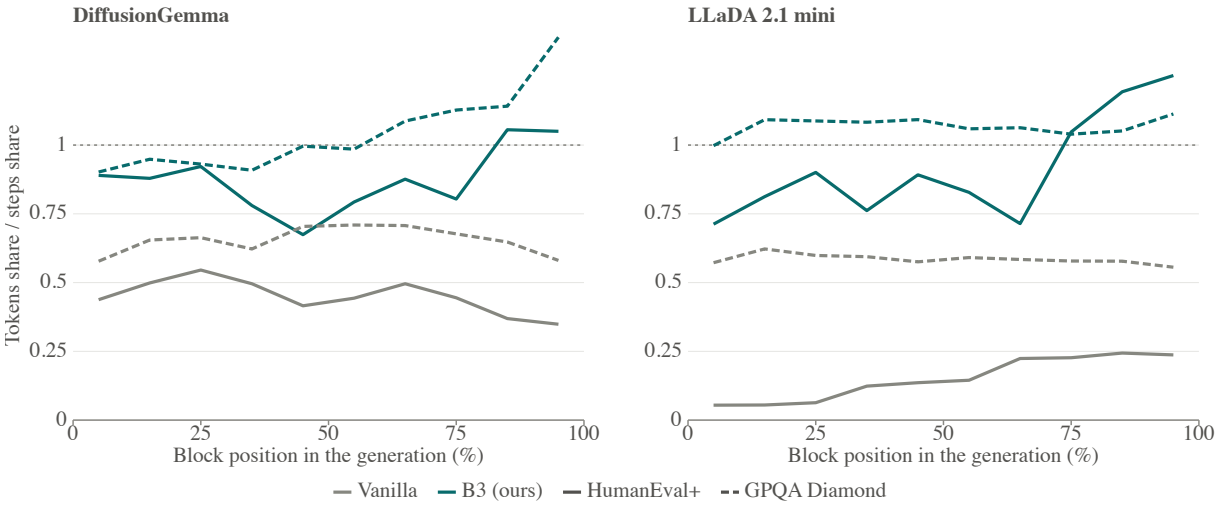

Figure 11: Share of a block's tokens accepted in its last 20% (more than 80% of the block accepted), divided by the share of its denoising steps taken there, by the block's position in the generation (first block 0%, last 100%; 10 bins). Below 1: the end of the block yields fewer tokens than the steps it costs. Solid: HumanEval+; dashed: GPQA Diamond. Dotted: parity.

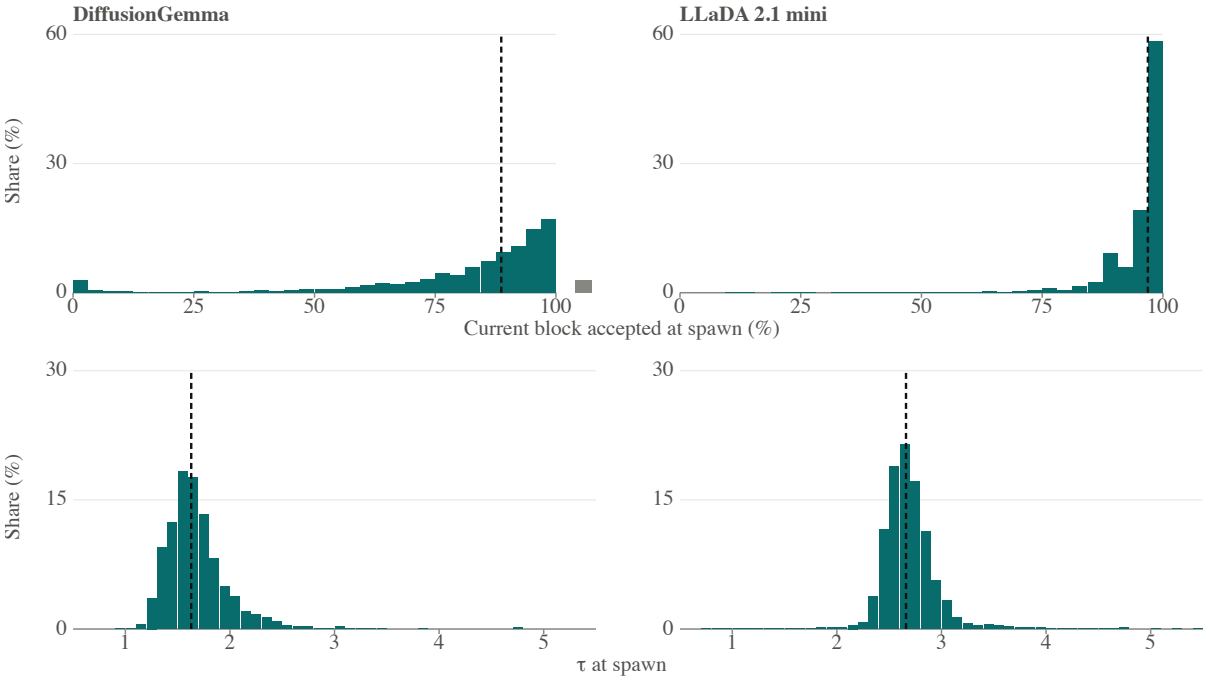

Figure 12: Spawn points of *B3*, pooled over HumanEval+ and GPQA Diamond. Top: share of the current block accepted when the next block spawns; gray: blocks that finished without a spawn. Bottom: τ at spawn. Dashed: median.

G DiffusionGemma Notation Clarifications

Two implementation details of DiffusionGemma (DiffusionGemma Team, 2026) differ slightly from the general description of block diffusion in Section 3.3. Neither changes the notation or the method.

Revoked acceptances. In DiffusionGemma, an accepted token can in principle be unaccepted on a later forward pass. Its position is then removed from the set of accepted positions and its token is replaced by noise drawn from π for the next forward pass. The set I_i of accepted positions, and hence the frontier $r_i = \max I_i$, are therefore not guaranteed to be monotone in i. This is a very rare occurrence, and the definitions of I_i, r_i and C_τ

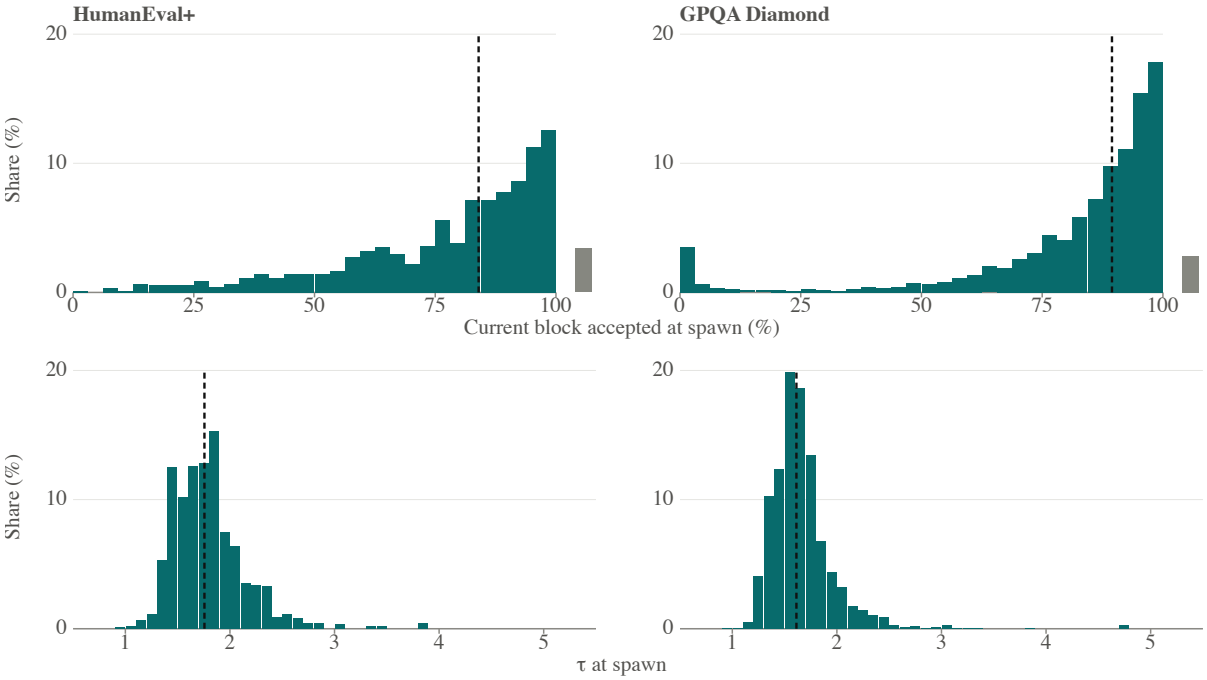

Figure 13: DiffusionGemma: spawn points per dataset. Top: share of the current block accepted at spawn; gray: blocks that finished without a spawn. Bottom: τ at spawn. Dashed: median.

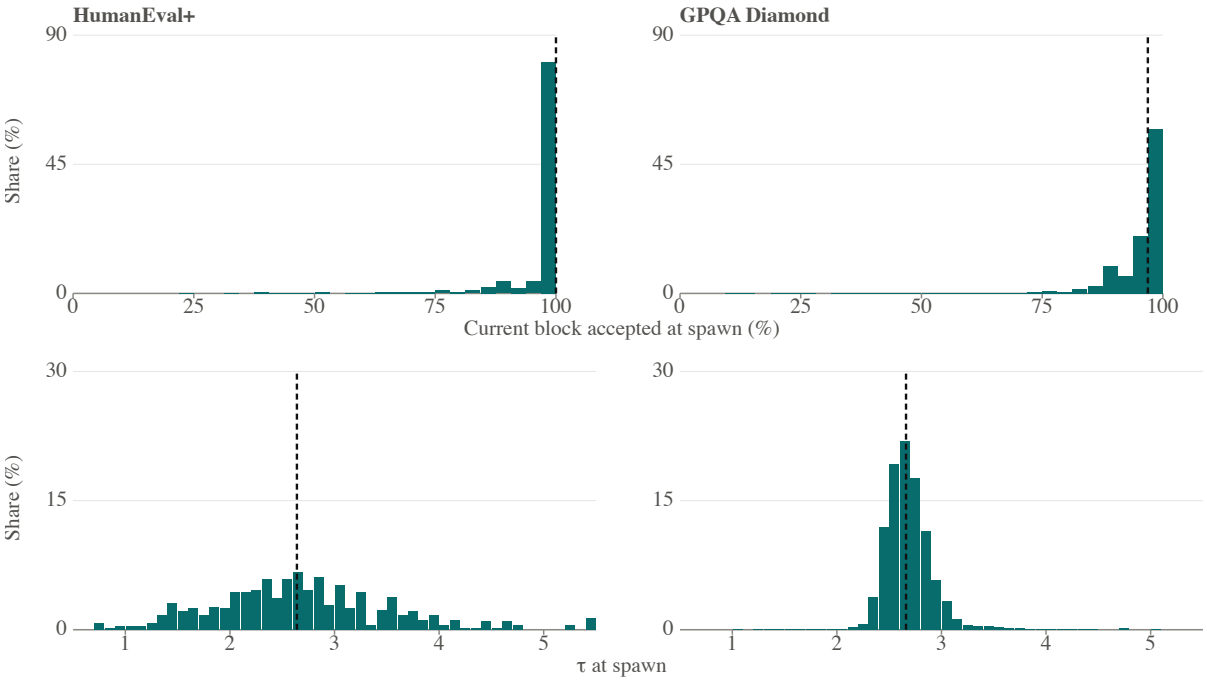

Figure 14: LLaDA 2.1 mini: spawn points per dataset. Top: share of the current block accepted at spawn; gray: blocks that finished without a spawn. Bottom: τ at spawn. Dashed: median.

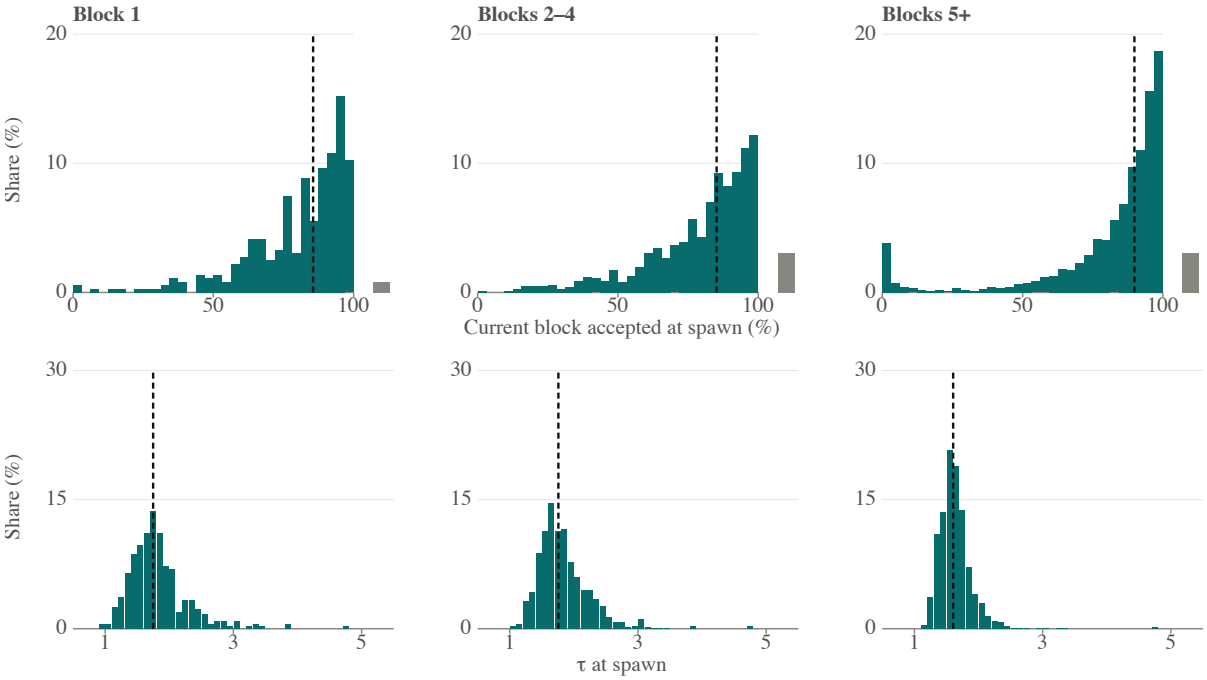

Figure 15: DiffusionGemma: spawn points per block index, pooled over datasets. Top: share of the current block accepted at spawn; gray: blocks that finished without a spawn. Bottom: τ at spawn. Dashed: median.

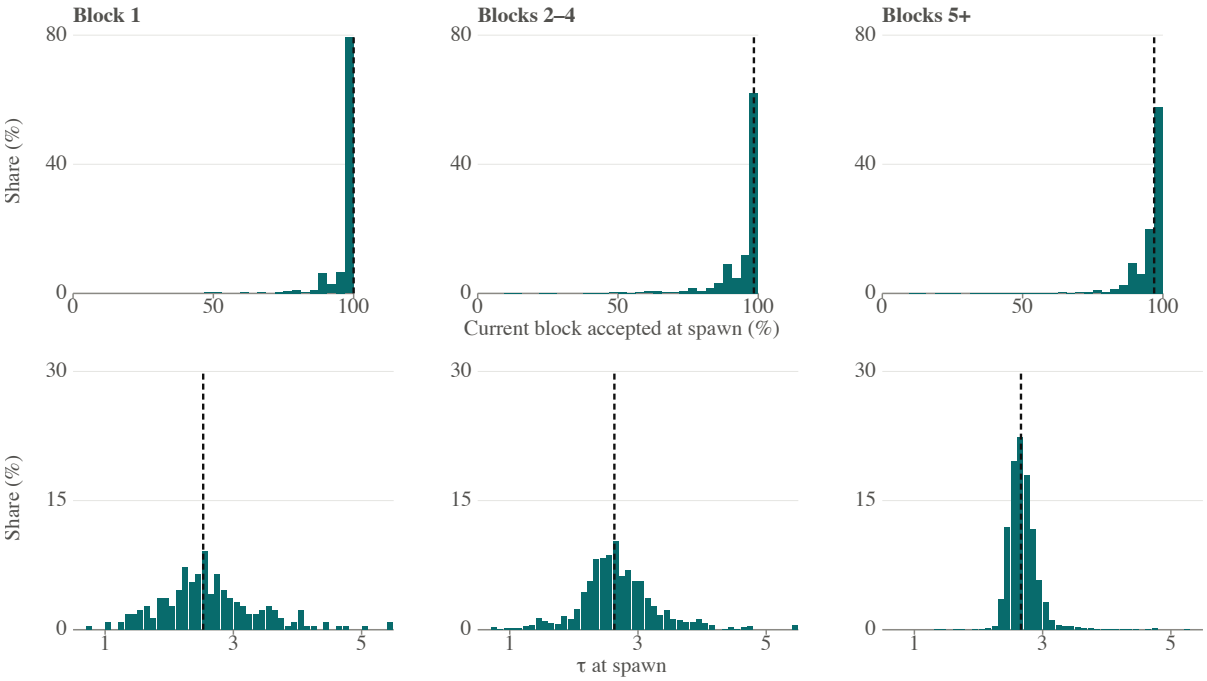

Figure 16: LLaDA 2.1 mini: spawn points per block index, pooled over datasets. Top: share of the current block accepted at spawn; gray: blocks that finished without a spawn. Bottom: τ at spawn. Dashed: median.

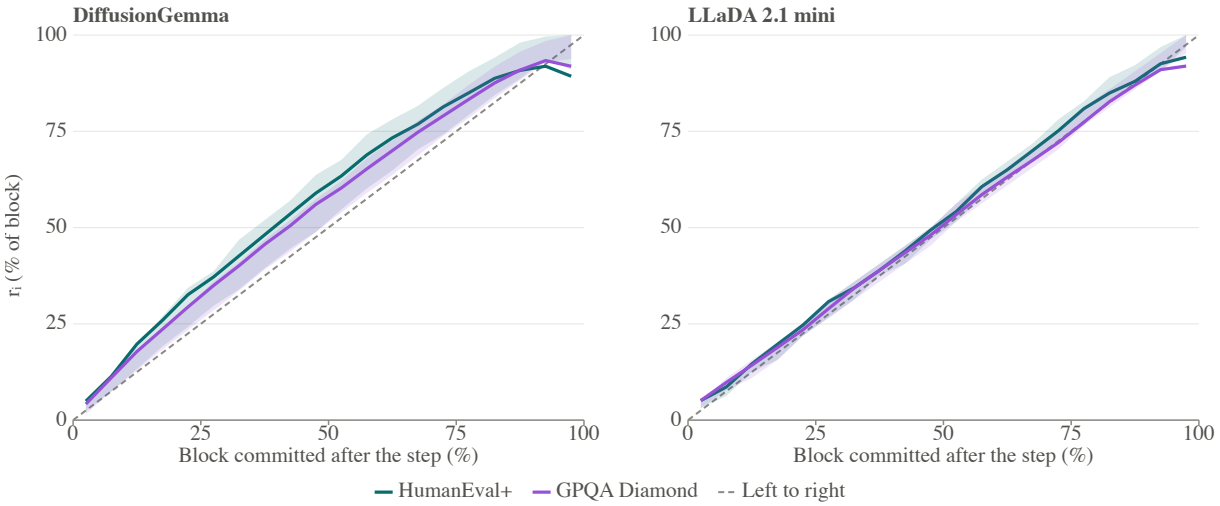

Figure 17: Rightmost position accepted on a denoising step against the share of the block committed after it, vanilla decoding. Lines: mean; bands: P25–P75; dashed: strictly left-to-right.

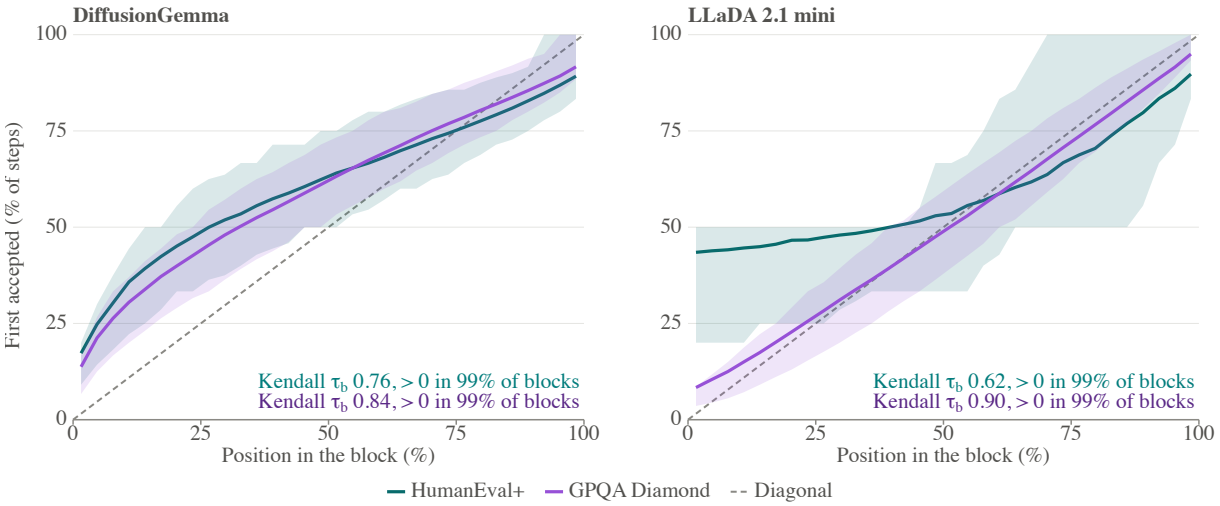

Figure 18: Step at which each position is first accepted, as a share of the block's denoising steps that accept at least one token, against its position in the block, vanilla decoding. Lines: mean; bands: P25–P75; dashed: diagonal. Kendall τ_b: median over blocks of the rank correlation between position and first-acceptance step, and the share of blocks where it is positive.

apply unchanged.

Causal history. DiffusionGemma encodes its history with fully causal attention, which does not change the forward pass of *B3*. Our method operates on DiffusionGemma's token-level causal KV cache, and only the current and future blocks attend block-causally (Figure 3). Once the current block is finalized (*committed*, in DiffusionGemma's terminology), it is re-encoded with token-level causal attention and pushed to the history, exactly as in the regular DiffusionGemma inference stack.

H LLM Use in Research

We used Claude Code (Anthropic, 2026b) for assistance with code writing and code reviewing, as well as LaTeX writing and figure generation. We also used Claude Chat (Anthropic, 2026a) for text drafting and as a reviewer of the paper.

Block	128	256	**256** ***B3***	384	512
TPS					
GSM8K	90.9	136.1	**150.8**	171.0	177.8
MATH500	96.6	139.9	**156.0**	167.8	171.1
AIME26	86.7	101.0	**121.6**	118.9	112.6
GPQA-D	70.4	94.4	**107.1**	106.0	104.8
IFEval	66.1	103.4	**119.9**	123.0	135.9
HumanEval+	99.9	148.9	**160.7**	172.3	188.8
MBPP+	90.5	136.4	**149.6**	165.2	183.7
LCB	78.1	108.0	**119.2**	118.7	118.8
Average	84.9	121.0	**135.6**	142.9	149.2
TPF					
GSM8K	18.59	28.74	**33.69**	36.62	40.96
MATH500	19.47	28.69	**34.16**	35.15	38.15
AIME26	16.43	20.61	**25.48**	24.55	24.70
GPQA-D	13.43	19.18	**22.88**	22.04	22.95
IFEval	13.53	21.08	**26.61**	26.67	30.54
HumanEval+	20.15	30.77	**37.53**	38.34	42.98
MBPP+	18.01	28.39	**33.40**	35.66	40.65
LCB	15.41	22.11	**25.59**	25.04	26.12
Average	16.88	24.95	**29.92**	30.51	33.38
Accuracy					
GSM8K	95.5	95.8	**96.1**	95.8	95.5
MATH500	96.4	95.8	**92.4**	94.6	93.2
AIME26	73.3	73.3	**76.7**	43.3	43.3
GPQA-D	73.2	73.7	**71.7**	73.7	65.7
IFEval	96.3	95.8	**95.4**	96.2	93.9
HumanEval+	94.5	91.5	**89.6**	90.9	91.5
MBPP+	81.7	80.7	**82.3**	79.1	80.4
LCB	71.1	70.5	**69.3**	68.1	62.0
Average	85.3	84.6	**84.2**	80.2	78.2

Table 5: DiffusionGemma 26B, per-benchmark results for Figure 5.

Block	32	**32** ***B3***	64	**64** ***B3***	96	128
TPS						
GSM8K	101.5	**122.8**	114.2	**124.8**	113.5	91.7
MATH500	133.9	**163.9**	160.4	**186.1**	159.7	148.5
AIME26	103.6	**118.5**	113.7	**125.9**	121.3	105.4
GPQA-D	65.8	**73.4**	69.5	**73.8**	70.5	62.9
IFEval	32.8	**35.4**	31.1	**32.4**	28.8	25.3
HumanEval+	204.2	**257.0**	247.2	**301.4**	249.6	188.4
MBPP+	142.3	**173.4**	163.6	**194.2**	168.3	138.6
LCB	105.1	**122.3**	113.7	**131.8**	115.1	93.0
Average	111.1	**133.3**	126.7	**146.3**	128.3	106.7
TPF						
GSM8K	4.36	**5.32**	4.88	**5.49**	4.86	4.05
MATH500	5.82	**7.16**	6.85	**8.14**	7.02	6.45
AIME26	4.41	**5.12**	4.92	**5.46**	5.10	4.46
GPQA-D	2.80	**3.18**	3.02	**3.26**	3.03	2.71
IFEval	1.41	**1.52**	1.35	**1.40**	1.24	1.09
HumanEval+	9.14	**11.52**	10.94	**13.69**	10.93	8.33
MBPP+	6.14	**7.73**	7.16	**8.57**	7.37	6.16
LCB	4.53	**5.45**	5.10	**5.83**	5.02	4.17
Average	4.83	**5.88**	5.53	**6.48**	5.57	4.68
Accuracy						
GSM8K	90.1	**90.4**	90.9	**89.8**	89.7	80.7
MATH500	91.6	**92.4**	92.8	**91.8**	90.2	85.8
AIME26	46.7	**56.7**	46.7	**50.0**	46.7	30.0
GPQA-D	52.5	**50.5**	52.5	**52.0**	46.5	45.5
IFEval	90.6	**89.3**	86.5	**86.0**	86.2	78.3
HumanEval+	82.9	**81.7**	83.5	**81.7**	75.6	61.0
MBPP+	70.4	**69.6**	69.8	**69.0**	68.8	70.9
LCB	25.7	**26.0**	25.7	**26.6**	24.6	20.5
Average	68.8	**69.6**	68.6	**68.4**	66.0	59.1

Table 6: LLaDA 2.1 mini 16B, per-benchmark results for Figure 5.

Block	32				64			
Method	Base	FlowBlock	MBD-LMs	***B3***	Base	FlowBlock	MBD-LMs	***B3***
TPS								
GSM8K	101.5	153.2	119.9	**122.8**	114.2	153.8	119.3	**124.8**
MATH500	133.9	207.1	161.8	**163.9**	160.4	210.8	176.6	**186.1**
AIME26	103.6	143.6	116.4	**118.5**	113.7	147.7	114.0	**125.9**
GPQA-D	65.8	91.1	74.2	**73.4**	69.5	83.4	69.7	**73.8**
IFEval	32.8	26.5	35.1	**35.4**	31.1	24.1	31.4	**32.4**
HumanEval+	204.2	227.9	253.3	**257.0**	247.2	236.0	299.7	**301.4**
MBPP+	142.3	118.7	167.8	**173.4**	163.6	116.9	195.5	**194.2**
LCB	105.1	85.1	120.5	**122.3**	113.7	86.0	128.6	**131.8**
Average	111.1	131.7	131.1	**133.3**	126.7	132.3	141.8	**146.3**
TPF								
GSM8K	4.36	6.66	5.32	**5.32**	4.88	6.74	5.33	**5.49**
MATH500	5.82	9.06	7.32	**7.16**	6.85	9.55	8.03	**8.14**
AIME26	4.41	6.27	5.18	**5.12**	4.92	6.51	5.06	**5.46**
GPQA-D	2.80	3.92	3.27	**3.18**	3.02	3.67	3.14	**3.26**
IFEval	1.41	1.15	1.55	**1.52**	1.35	1.04	1.38	**1.40**
HumanEval+	9.14	10.24	11.72	**11.52**	10.94	10.62	13.70	**13.69**
MBPP+	6.14	5.20	7.74	**7.73**	7.16	5.24	8.69	**8.57**
LCB	4.53	3.67	5.46	**5.45**	5.10	3.78	5.87	**5.83**
Average	4.83	5.77	5.95	**5.88**	5.53	5.89	6.40	**6.48**
Accuracy								
GSM8K	90.1	90.4	90.4	**90.4**	90.9	91.1	88.2	**89.8**
MATH500	91.6	91.8	90.0	**92.4**	92.8	91.6	88.6	**91.8**
AIME26	46.7	36.7	40.0	**56.7**	46.7	40.0	40.0	**50.0**
GPQA-D	52.5	49.0	50.0	**50.5**	52.5	50.0	41.4	**52.0**
IFEval	90.6	90.5	89.3	**89.3**	86.5	85.1	84.4	**86.0**
HumanEval+	82.9	84.1	82.9	**81.7**	83.5	84.1	79.9	**81.7**
MBPP+	70.4	72.0	71.2	**69.6**	69.8	74.6	72.8	**69.0**
LCB	25.7	28.4	25.4	**26.0**	25.7	29.8	27.5	**26.6**
Average	68.8	67.9	67.4	**69.6**	68.6	68.3	65.3	**68.4**

Table 7: LLaDA 2.1 mini 16B, per-benchmark results for Table 2.

Accepted in	Tokens	Edited later	Share
DiffusionGemma			
Vanilla	237,496	633	0.27%
B3 (future block)	42,426	839	1.98%
LLaDA 2.1 mini			
Vanilla	51,520	1,441	2.80%
B3 (future block)	26,250	1,125	4.29%

Table 8: Tokens edited after their first acceptance, on HumanEval+.